

Project Documentation: VeriRISC Generic Counter Unit

Digital Design Lab

Project Overview

This lab implements a parameterized **Generic Counter Unit** for the VeriRISC CPU in Verilog HDL [?]. The VeriRISC CPU utilizes this reusable counter module to serve as both the program counter (PC) and the phase counter [?]. The module features asynchronous active-high reset (**rst**), synchronous parallel load (**load**), and counting enable (**enab**) capabilities [?, ?, ?].

Specifications & Functionality

- **Data Width (WIDTH):** Parameterized to 5 bits by default ($2^5 = 32$ states) [?, ?, ?].
- **Asynchronous Reset:** Active-high reset (**rst**) immediately forces **cnt_out** to zero on posedge **rst** [?, ?, ?].
- **Synchronous Load:** Loads parallel input **cnt_in** directly to **cnt_out** on posedge **clk** when **load** = 1 [?, ?, ?].
- **Enable / Increment:** Increments **cnt_out** by 1 on posedge **clk** when **load** = 0 and **enab** = 1 [?, ?, ?].
- **Hold Value:** Retains current **cnt_out** state when neither reset, load, nor enable are active [?, ?, ?].

Port Summary

Port Name	Direction	Type	Width	Description
clk	Input	wire	1	Clock input signal [?, ?, ?]
rst	Input	wire	1	Asynchronous active-high reset [?, ?, ?]
load	Input	wire	1	Synchronous parallel load enable [?, ?, ?]
enab	Input	wire	1	Counter increment enable [?, ?, ?]
cnt_in	Input	wire	WIDTH (5)	Parallel load data input [?, ?, ?]
cnt_out	Output	reg	WIDTH (5)	Counter output value [?, ?, ?]

Table 1: Counter Module Port Interfaces

Verilog Source Code

Design Module (`counter.v`)

```

module counter #(
    parameter WIDTH = 5
)(
    input  wire      clk,
    input  wire      rst,
    input  wire      load,
    input  wire      enab,
    input  wire [WIDTH-1:0] cnt_in,
    output reg  [WIDTH-1:0] cnt_out
);

    // Sequential process with asynchronous reset
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            cnt_out <= {WIDTH{1'b0}};
        end else if (load) begin
            cnt_out <= cnt_in;
        end else if (enab) begin
            cnt_out <= cnt_out + 1'b1;
        end
    end
endmodule

```

Testbench Module (counter_test.v)

```

`timescale 1ns / 1ps

module counter_test;

    localparam WIDTH = 5;

    reg      clk;
    reg      rst;
    reg      load;
    reg      enab;
    reg [WIDTH-1:0] cnt_in;
    wire [WIDTH-1:0] cnt_out;

    counter #(
        .WIDTH ( WIDTH )
    ) counter_inst (
        .clk      ( clk      ),
        .rst      ( rst      ),
        .load     ( load     ),
        .enab     ( enab     ),
        .cnt_in   ( cnt_in   ),
        .cnt_out  ( cnt_out  )
    );

    task expect;
        input [WIDTH-1:0] exp_out;
        if (cnt_out != exp_out) begin
            $display("TEST FAILED");
        end
    endtask

```

```

    $display("At time %0d rst=%b load=%b enab=%b cnt_in=%b cnt_out=%b",
            $time, rst, load, enab, cnt_in, cnt_out);
    $display("cnt_out should be %b", exp_out);
end else begin
    $display("At time %0d rst=%b load=%b enab=%b cnt_in=%b cnt_out=%b",
            $time, rst, load, enab, cnt_in, cnt_out);
end
end
endtask

initial repeat (7) begin #5 clk=1; #5 clk=0; end

initial @(negedge clk) begin
    rst=0; load=1; enab=1; cnt_in=5'h15; @(negedge clk) expect (5'h15);
    rst=0; load=1; enab=1; cnt_in=5'h0A; @(negedge clk) expect (5'h0A);
    rst=0; load=1; enab=1; cnt_in=5'h1F; @(negedge clk) expect (5'h1F);
    rst=1; load=1; enab=1; cnt_in=5'h1F; @(negedge clk) expect (5'h00);
    rst=0; load=1; enab=1; cnt_in=5'h1F; @(negedge clk) expect (5'h1F);
    rst=0; load=0; enab=1; cnt_in=5'h1F; @(negedge clk) expect (5'h00);
    $display("TEST PASSED");
    $finish;
end

endmodule

```

Simulation & Verification

Execution Command

To run batch simulation using Cadence Xcelium [?]:

```
xrun counter.v counter_test.v
```

Expected Simulation Output

The output log verifies correct execution across all clock edges and control precedence conditions [?]:

```

At time 20 rst=0 load=1 enab=1 cnt_in=10101 cnt_out=10101
At time 30 rst=0 load=1 enab=1 cnt_in=01010 cnt_out=01010
At time 40 rst=0 load=1 enab=1 cnt_in=11111 cnt_out=11111
At time 50 rst=1 load=1 enab=1 cnt_in=11111 cnt_out=00000
At time 60 rst=0 load=1 enab=1 cnt_in=11111 cnt_out=11111
At time 70 rst=0 load=0 enab=1 cnt_in=11111 cnt_out=00000
TEST PASSED

```